LECTURE 02

02

Fundamentals of Algorithmic Problem Solving

How to understand, design, implement, and refine a computational solution systematically.

The focus today is not faster code. It is better thinking before code.

LEARNING OUTCOMES

What should you be able to do by the end?

Lecture 2 follows the CSC301 CDF: STAIR, problem specification, constraints, resources, and solution selection.

STAIR

apply a systematic problem-solving framework

SPECIFY

state input, output, constraints, and assumptions

UNDERSTAND

use examples and edge cases to clarify the task

ANALYZE

recognize factors that influence algorithm choice

RESOURCES

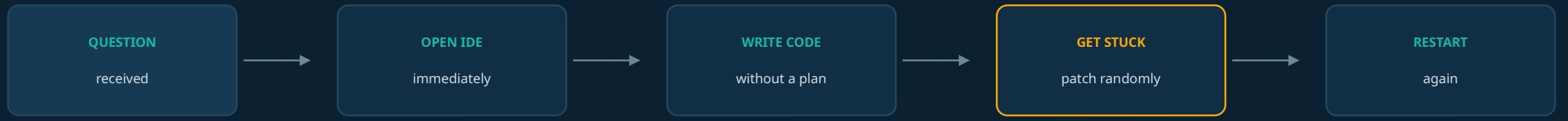
consider time, memory, hardware, and query frequency

COMPARE

identify multiple approaches to the same problem

You already know how to code...

But coding is not the first step.



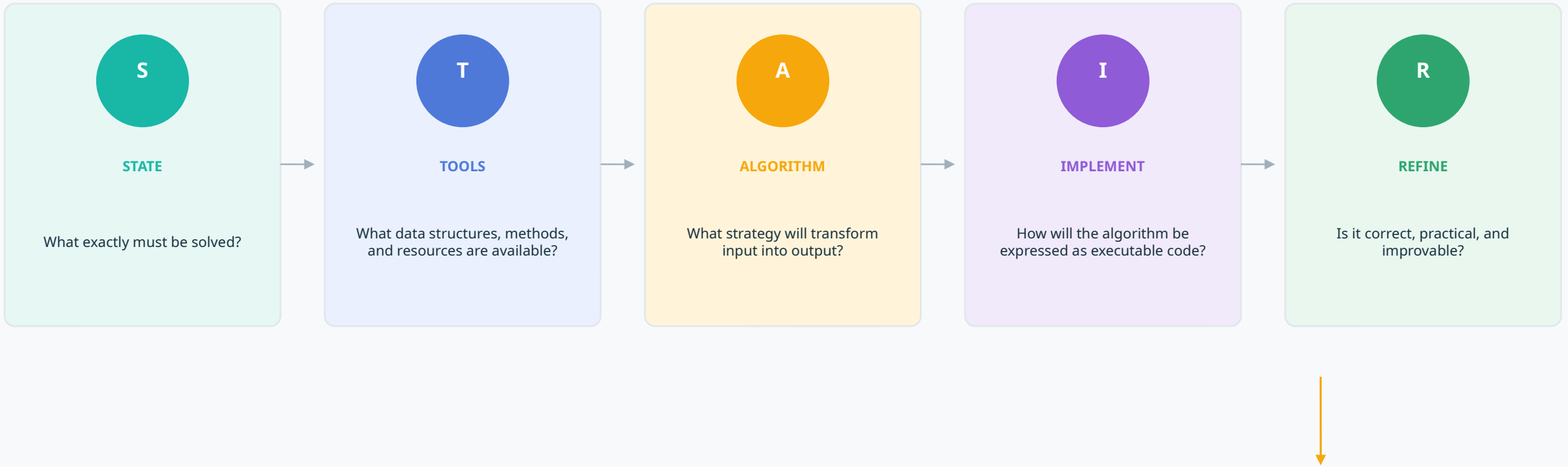
Better sequence **UNDERSTAND & SPECIFY → CHOOSE TOOLS → DESIGN → IMPLEMENT → REFINE**

A large fraction of “coding problems” begin as problem-definition problems.

FRAMEWORK

STAIR: a compact problem-solving discipline

State the problem → Tools for the job → Algorithm development → Implementation → Refinement



Do not solve a vague request

A useful specification removes ambiguity before design begins.

VAGUE REQUEST

“Find a student.”

COMPUTATIONAL SPECIFICATION

Given an array A of n student registration numbers and a target registration number x, return the position of x if present; otherwise return NOT FOUND.

Questions hidden inside the vague version

By name or ID? • One match or all? • What if absent? • Is the data sorted?

Four things to pin down before designing

A computational problem becomes manageable when its contract is explicit.

INPUT

A = student IDs
x = target ID

What data is supplied

OUTPUT

index of x
or NOT FOUND

What must be returned

CONSTRAINTS

n may be large
IDs may be sorted

What assumptions apply

VALIDITY

returned index points to x
or x is truly absent

When is the answer correct

The quality of the algorithm cannot exceed the quality of the problem statement.

Use small examples and edge cases before writing pseudocode

Manual reasoning often exposes hidden assumptions.

NORMAL CASE

A = [4, 9, 2, 7]
x = 2

4 ≠ 2
9 ≠ 2
2 = 2 → found at index 2

EDGE CASES

- empty array
- target is first element
- target is last element
- target is absent
- duplicate targets

QUESTIONS TO ASK

What is valid input?
What should happen when no match exists?
If duplicates exist, return first, last, or all?

The same values can permit different algorithms

Input properties are information an algorithm may exploit.

UNSORTED INPUT

72 13 91 4 55 28 80

SORTED INPUT

4 13 28 55 72 80 91

Other characteristics worth checking

- input size
- sorted / unsorted
- duplicates
- bounded values
- static / changing
- dense / sparse

A useful property can change the algorithmic toolbox.

Small wording changes create different problems

Define exactly what must be produced—and under what limits.

SAME DATA, DIFFERENT OUTPUT

Does x exist? → TRUE / FALSE

Where is x? → index

Return x record → student object

Find every “Ali” → collection of records

CONSTRAINTS THAT CHANGE DESIGN

$n \leq 20$ vs. $n = 100{,}000{,}000$

response needed in 5 ms

memory limited to 1 MB

data changes every second

search runs once vs. 50,000 times/day

Your algorithmic toolbox grows throughout the course

“Tool” can mean a data structure, a known algorithm, a design technique, or a computational resource.

DATA STRUCTURES

Array
Linked list
Stack / Queue
Tree
Graph
Hash table
Heap

KNOWN ALGORITHMS

Linear search
Binary search
Sorting methods
Graph traversal
Shortest paths
…

DESIGN TECHNIQUES

Brute force
Divide and conquer
Decrease and conquer
Dynamic programming
Greedy
…

Algorithms run on real machines, not infinite ones

Practical solutions must live within computational and operational limits.

TIME

response deadline
real-time vs. batch

MEMORY

working storage
RAM / cache / device limits

HARDWARE

computational device
desktop / server / embedded

COMMUNICATION

data movement
network bandwidth / latency

ENERGY

power budget
mobile / sensor / edge device

FREQUENCY

how often used
one query vs. millions

Device capability and exactness matter

Levitin explicitly asks us to consider the computational device and whether an exact answer is required.

COMPUTATIONAL DEVICE

128 GB server RAM
≠
128 KB embedded memory

EXACT vs. APPROXIMATE

Exact: locate student ID 127
Approximate: near-optimal route under a tight deadline

The “best” solution is always relative to the problem specification and the resources available.

Turn a clear problem into a strategy

Start with a simple idea, then express it as a finite procedure.

SEARCHING PROBLEM

A = [27, 14, 81, 3, 45, 62]
x = 45

Goal: return x's index or NOT FOUND.

SIMPLE STRATEGY

Start at the beginning.
Compare each element with x.
Stop as soon as a match is found.
If the end is reached, report NOT FOUND.

```
LINEAR-SEARCH(A, x)
for i ← 0 to n − 1
   if A[i] = x
      return i
return NOT-FOUND
```

REPRESENTATIVE EXAMPLE

Trace Linear Search before discussing alternatives

Tracing verifies what the procedure actually does on a concrete instance.



Return index 4

One problem can have several reasonable strategies

The right choice depends on input properties and usage patterns.

LINEAR SEARCH

works on unsorted data

simple; inspect sequentially

BINARY SEARCH

requires sorted data

discard a large region each step

HASH LOOKUP

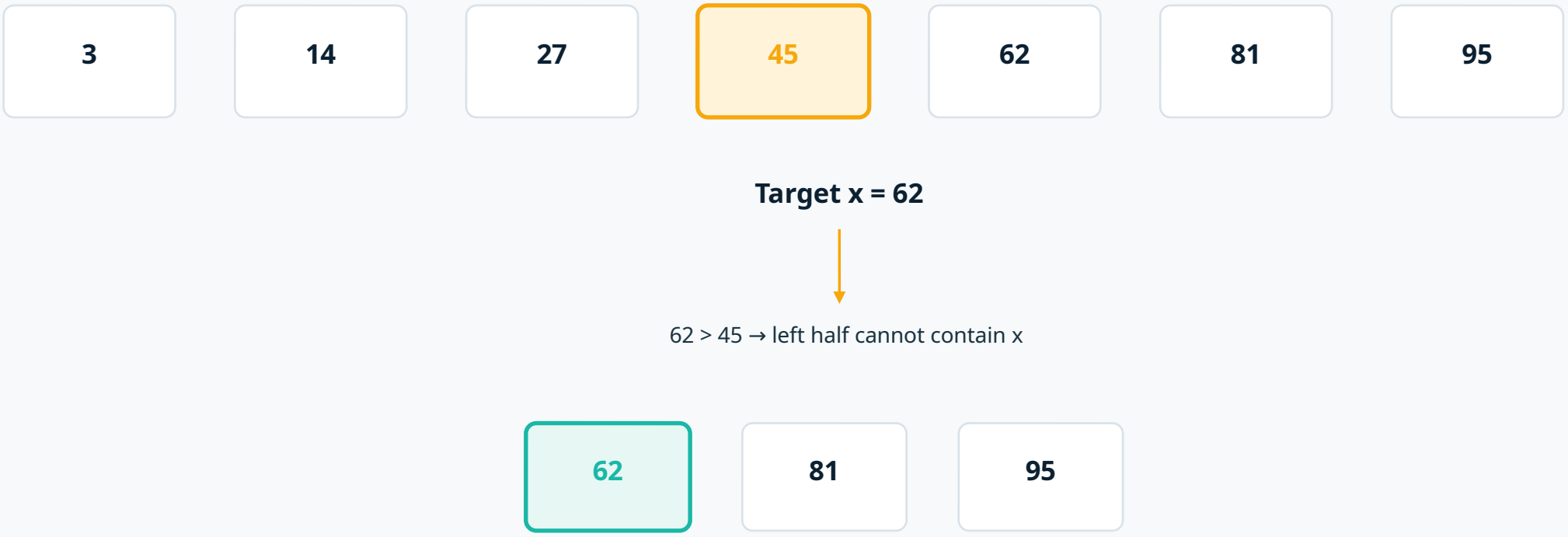
requires extra organization

useful for many repeated lookups

Today: compare them conceptually. Formal efficiency analysis comes later in the CDF.

Binary Search: repeatedly narrow the search region

No asymptotic notation yet—focus on the design idea.



One search vs. fifty thousand searches per day

Sometimes we spend effort before the query to make future queries cheaper.

CASE A

20 unsorted values
One search

A simple sequential scan may be perfectly reasonable.

CASE B

1,000,000 sorted IDs
One search

Binary Search can exploit the existing order.

CASE C

1,000,000 records
50,000 searches/day
Extra memory available

Indexing / hashing may justify preprocessing.

Translate the algorithm into an executable form

Implementation should preserve the algorithmic logic.

PSEUDOCODE

```
LINEAR-SEARCH(A, x)

for i ← 0 to n − 1
    if A[i] = x
        return i

return −1
```

C++ IMPLEMENTATION

```
int linearSearch(const vector<int>& A, int x)
{
    for (int i = 0; i < A.size(); ++i)
    {
        if (A[i] == x)
            return i;
    }
    return -1;
}
```

Algorithm = solution logic. Implementation = one language-specific realization.

A working program is not automatically the end

Refinement means testing the solution against the original problem and reconsidering earlier choices.

CORRECT?

Does it return the required result?

COMPLETE?

Empty input? absent target? duplicates?

PRACTICAL?

Acceptable time and memory?

SCALABLE?

What happens when n becomes large?

BETTER TOOL?

Can input properties be exploited?

Refine → revisit State, Tools, Algorithm, or Implementation

STAIR is the memory aid; Levitin gives the richer design checklist

The two fit together naturally, but they are not the same named framework.

S — State	Understand the problem
T — Tools	Device capability • exact vs. approximate • design techniques • data structures
A — Algorithm	Design the algorithm • specify it clearly
I — Implementation	Code the algorithm
R — Refinement	Check correctness • analyze • reconsider / improve

Search for a student registration number

Walk through the framework without jumping directly to code.

S — Input A, target x; return index or NOT FOUND. Ask: sorted? large? frequent queries?

T — Linear search, binary search, hashing/indexing; available memory and preprocessing time.

A — Choose a strategy that fits the actual input and usage pattern.

I — Express the chosen procedure in pseudocode, then implement it.

R — Test absent/edge cases; reassess time, memory, and whether another tool is preferable.

Key idea: there is no universally best algorithm independent of the problem conditions.

Problem 1 — Find a Student ID

Same broad goal; input organization and query frequency change the sensible solution.

STAIR	CASE A • SMALL / ONCE	CASE B • LARGE / SORTED	CASE C • LARGE / REPEATED
S — STATE	Find target x among 30 unsorted IDs.	Find target x among 1,000,000 sorted IDs.	Find target x among 1,000,000 records, 50,000 times/day.
T — TOOLS	Array.	Sorted array.	Hash table / index.
A — ALGORITHM	Linear search.	Binary search.	Hash lookup.
I — IMPLEMENT	Loop through IDs one by one.	Maintain low, high, and mid.	Build the index once; then perform lookups.
R — REFINE	Simple solution is sufficient.	Exploit the existing sorted order.	Extra memory and preprocessing are justified.

Same goal ≠ same best algorithm. Conditions determine the choice.

Problem 2 — Serve People Waiting

A change in service rules can force a different data structure and scheduling algorithm.

STAIR	CASE A • CAFETERIA	CASE B • EMERGENCY	CASE C • VIP + NORMAL
S — STATE	Serve students in arrival order.	Serve patients according to urgency.	Serve VIP customers before normal customers.
T — TOOLS	Queue.	Priority queue.	Two queues / priority queue.
A — ALGORITHM	FIFO: first arrival is served first.	Highest-priority patient first.	Serve VIP queue first, then normal queue.
I — IMPLEMENT	enqueue() and dequeue().	Priority field + heap / priority queue.	Separate queues or a priority field.
R — REFINE	Normal queue works.	FIFO is no longer suitable.	Define fairness and tie-breaking rules.

Refinement may reveal that the tool—not the code—is the wrong choice.

Problem 3 — Find a Route

The same map supports different algorithms depending on what “best route” means.

STAIR	CASE A • ANY ROUTE	CASE B • FEWEST ROADS	CASE C • SHORTEST DISTANCE
S — STATE	Find any valid route from A to B.	Find a route using the fewest road segments.	Find the minimum-distance route from A to B.
T — TOOLS	Graph.	Graph + queue.	Weighted graph + priority queue.
A — ALGORITHM	DFS or BFS.	BFS.	Dijkstra’s algorithm.
I — IMPLEMENT	Adjacency list + traversal.	Queue + visited nodes.	Distance array + min-priority queue.
R — REFINE	Any valid path is enough.	Count edges, not physical distance.	Road weights must represent distance correctly.

Changing the meaning of “best” changes the algorithmic problem.

Problem 4 — Detect Duplicate Student IDs

Multiple correct approaches exist; memory and input size determine which one is practical.

STAIR	CASE A • 20 IDs	CASE B • LARGE / LOW MEMORY	CASE C • LARGE / MEMORY OK
S — STATE	Determine whether any duplicate ID exists.	Same goal for a very large dataset with limited memory.	Same goal for a very large dataset with extra memory.
T — TOOLS	Array.	Array + sorting.	Hash set.
A — ALGORITHM	Compare every pair.	Sort, then compare adjacent IDs.	Insert/check each ID in the set.
I — IMPLEMENT	Nested loops.	Sorting + one sequential scan.	unordered_set / hash structure.
R — REFINE	Simplicity is reasonable at tiny scale.	Avoid extra hash memory.	Use memory to avoid repeated searching.

Correctness alone does not choose among several valid algorithms.

Problem 5 — Highest Marks / Top Students

A small change in required output can change what must be stored and how it is processed.

STAIR	CASE A • MAX ONCE	CASE B • STREAMING	CASE C • TOP 10
S — STATE	Find the highest mark in a completed class list.	Maintain the highest mark as results arrive continuously.	Find the ten highest-scoring students.
T — TOOLS	Array + max variable.	Single max variable / stream.	Min-heap of size 10.
A — ALGORITHM	Scan once and update maximum.	Compare each new mark with current maximum.	Maintain the best 10 while scanning.
I — IMPLEMENT	for loop over stored marks.	Update max on each arrival; no full list required.	Insert into heap; remove smallest when size > 10.
R — REFINE	Simple scan is enough.	Do not store data you do not need.	Heap avoids repeatedly sorting the full dataset.

Output requirements influence both the tool and the algorithm.

CLASS ACTIVITY

Transfer the STAIR reasoning to a new case

Do not name an algorithm first. Identify the requirement, input property, or constraint that drives the choice.

CASE A

20 unsorted numbers
One search

What is the simplest sensible approach?

CASE B

1,000,000 sorted IDs
One search

Which property should you exploit?

CASE C

1,000,000 records
50,000 searches/day
Extra memory available

Would preprocessing or indexing be justified?

Your answer must mention input characteristics, constraints, and expected usage.

Recommended textbook exercises

Use the question numbers below from Levitin, 3rd edition, Exercises 1.2.

CORE EXERCISES

Q1 — Old World river-crossing puzzle
Apply disciplined problem understanding and state constraints explicitly.

Q2 — Bridge-and-flashlight puzzle
Compare possible move sequences and justify a feasible plan.

Q9 — Improve the given minimum-distance algorithm
Look for implementation improvements and, if necessary, a better overall algorithm.

OPTIONAL / DISCUSSION

Q10 — Compare Pólya's four-step problem-solving plan with the process in Levitin §1.2.

Suggested submission focus:
• problem statement
• available tools / assumptions
• algorithm idea
• refinement opportunities

Do not use formal asymptotic analysis yet unless asked.

Reading: Levitin §1.2 — Fundamentals of Algorithmic Problem Solving

LECTURE 02

Five ideas to carry forward

1. State the problem before designing a solution.
2. Input characteristics and output requirements matter.
3. Algorithm choice depends on constraints and available resources.
4. Several correct approaches may exist for the same problem.
5. Refinement can send us back to any earlier STAIR stage.

NEXT

Algorithm Correctness Fundamentals

Preconditions • postconditions
partial vs. total correctness